

Лабораторная работа № 5

Bash-автоматизация проверки состояния Linux-сервера

Продолжительность

90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - установленная служба Nginx;
 - Bash 5.x;
 - текстовый редактор Nano или Vim.
-

1. Цель работы

Разработать административный Bash-сценарий:

`/usr/local/sbin/server-health.sh`

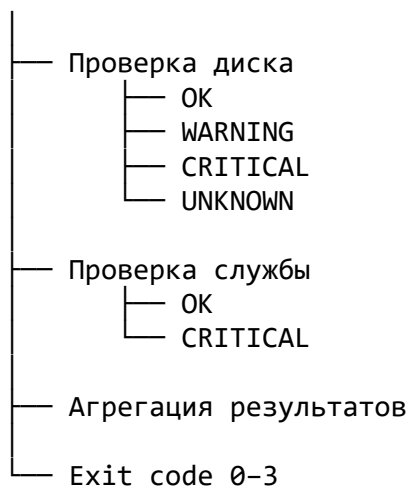
Сценарий должен:

- проверять заполнение файловой системы;
 - проверять состояние systemd-службы;
 - принимать параметры командной строки;
 - проверять корректность пороговых значений;
 - возвращать коды 0–3;
 - выводить понятные сообщения;
 - записывать результаты в Journald;
 - безопасно работать при повторном запуске;
 - корректно запускаться в минимальном окружении.
-

2. Итоговая логика сценария

Запуск

- Разбор аргументов
- Проверка порогов
- Проверка зависимостей



3. Коды завершения

В лабораторной работе используется соглашение, совместимое с подходом мониторинговых систем Nagios и Icinga.

Код	Состояние	Значение
0	OK	Проверки выполнены успешно
1	WARNING	Параметр достиг предупредительного порога
2	CRITICAL	Обнаружено критическое состояние
3	UNKNOWN	Сценарий не смог корректно выполнить проверку

Приоритет итоговых состояний:

CRITICAL → UNKNOWN → WARNING → OK

Например, если диск имеет состояние WARNING, а служба остановлена, итоговый код должен быть 2, то есть CRITICAL.

4. Параметры запуска

Сценарий должен поддерживать:

Параметр	Назначение
-h	Показать справку
-s SERVICE	Указать проверяемую systemd-службу
-w WARN	Указать порог WARNING
-c CRIT	Указать порог CRITICAL

Пример:

```
sudo /usr/local/sbin/server-health.sh \  
-s nginx \  
-w 80 \  
-c 90
```

Значения по умолчанию:

```
SERVICE=nginx  
MOUNTPPOINT=/  
DISK_WARN=80  
DISK_CRIT=90
```

5. Базовые задания

Задание 1. Подготовить сервер

Обновляем информацию о пакетах:

```
sudo apt update
```

Устанавливаем Nginx:

```
sudo apt install -y nginx
```

Проверяем службу:

```
systemctl is-active nginx  
systemctl status nginx --no-pager
```

Ожидаемое состояние:

```
active
```

Проверяем текущую загрузку корневой файловой системы:

```
df -P /
```

Пример:

Filesystem	1024-blocks	Used	Available	Capacity	Mounted on
/dev/sda2	25000000	7200000	16500000	31%	/

Проверяем команды, которые будут использоваться сценарием:

```
command -v bash  
command -v df  
command -v awk  
command -v systemctl  
command -v hostname  
command -v logger
```

Каждая команда должна вывести полный путь к исполняемому файлу.

Задание 2. Создать файл сценария

Создаём файл:

```
sudo nano /usr/local/sbin/server-health.sh
```

Добавляем начальный каркас:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'
export PATH
```

```
readonly EXIT_OK=0
readonly EXIT_WARNING=1
readonly EXIT_CRITICAL=2
readonly EXIT_UNKNOWN=3
```

```
SERVICE='nginx'
MOUNTPPOINT='/'
DISK_WARN=80
DISK_CRIT=90
```

```
main() {
    printf 'Сценарий запущен\n'
}
```

```
main "$@"
```

Сохраняем файл:

Ctrl+O
Enter
Ctrl+X

Проверяем синтаксис без выполнения:

```
sudo bash -n /usr/local/sbin/server-health.sh
```

Если ошибок нет, команда ничего не выводит.

Назначаем владельца и права:

```
sudo chown root:root /usr/local/sbin/server-health.sh
sudo chmod 750 /usr/local/sbin/server-health.sh
```

Проверяем:

```
ls -l /usr/local/sbin/server-health.sh
```

Пример:

```
-rwxr-x--- 1 root root ... /usr/local/sbin/server-health.sh
```

Задание 3. Добавить функцию справки

Перед функцией `main()` добавляем:

```
usage() {  
    cat <<EOF
```

Использование:

```
$0 [-h] [-s SERVICE] [-w WARN] [-c CRIT]
```

Параметры:

-h	показать справку
-s SERVICE	проверяемая systemd-служба
-w WARN	порог WARNING в процентах
-c CRIT	порог CRITICAL в процентах

Значения по умолчанию:

```
SERVICE=$SERVICE  
MOUNTPPOINT=$MOUNTPPOINT  
WARN=$DISK_WARN  
CRIT=$DISK_CRIT
```

EOF

```
}
```

Временно изменяем `main()`:

```
main() {  
    usage  
}
```

Проверяем:

```
sudo bash -n /usr/local/sbin/server-health.sh  
sudo /usr/local/sbin/server-health.sh
```

Обратить внимание

В сценарии используется:

```
main "$@"
```

Конструкция `"$@"` передаёт каждый исходный аргумент отдельно.

Конструкция `"$*"` объединила бы все аргументы в одну строку.

Задание 4. Добавить разбор аргументов

Добавляем функцию:

```
parse_arguments() {
    local opt

    OPTIND=1

    while getopts ':hs:w:c:' opt; do
        case "$opt" in
            h)
                usage
                exit "$EXIT_OK"
                ;;
            s)
                SERVICE=$OPTARG
                ;;
            w)
                DISK_WARN=$OPTARG
                ;;
            c)
                DISK_CRIT=$OPTARG
                ;;
            :)
                printf 'UNKNOWN: параметр -%s требует значение\n' \
                    "$OPTARG" >&2
                exit "$EXIT_UNKNOWN"
                ;;
            \?)
                printf 'UNKNOWN: неизвестный параметр -%s\n' \
                    "$OPTARG" >&2
                usage >&2
                exit "$EXIT_UNKNOWN"
                ;;
        esac
    done

    shift "$((OPTIND - 1))"

    if (( $# > 0 )); then
        printf 'UNKNOWN: лишние аргументы: %s\n' "$*" >&2
        exit "$EXIT_UNKNOWN"
    fi
}
```

Изменяем main():

```
main() {
    parse_arguments "$@"

    printf 'service=%s\n' "$SERVICE"
```

```

    printf 'warning=%s\n' "$DISK_WARN"
    printf 'critical=%s\n' "$DISK_CRIT"
}

```

Проверяем:

```

sudo /usr/local/sbin/server-health.sh \
-s ssh \
-w 70 \
-c 85

```

Ожидаемый результат:

```

service=ssh
warning=70
critical=85

```

Проверяем справку:

```

sudo /usr/local/sbin/server-health.sh -h
echo $?

```

Ожидаемый код:

```
0
```

Проверяем неизвестный параметр:

```

sudo /usr/local/sbin/server-health.sh -z
echo $?

```

Ожидаемый код:

```
3
```

Задание 5. Добавить проверку порогов

Добавляем функцию:

```

validate_percent() {
    local value=$1

    [[ $value =~ ^[0-9]+$ ]] || return 1
    (( value >= 1 && value <= 100 ))
}

```

Добавляем общую проверку параметров:

```

validate_arguments() {
    if [[ -z $SERVICE ]]; then
        printf 'UNKNOWN: имя службы не может быть пустым\n' >&2
        return "$EXIT_UNKNOWN"
    fi
}

```

```

if ! validate_percent "$DISK_WARN"; then
    printf 'UNKNOWN: WARNING должен быть числом от 1 до 100\n' >&2
    return "$EXIT_UNKNOWN"
fi

if ! validate_percent "$DISK_CRIT"; then
    printf 'UNKNOWN: CRITICAL должен быть числом от 1 до 100\n' >&2
    return "$EXIT_UNKNOWN"
fi

if (( DISK_WARN >= DISK_CRIT )); then
    printf 'UNKNOWN: WARNING должен быть меньше CRITICAL\n' >&2
    return "$EXIT_UNKNOWN"
fi

return "$EXIT_OK"
}

```

Изменяем main():

```

main() {
    parse_arguments "$@"

    local validation_rc=0
    validate_arguments || validation_rc=$?

    if (( validation_rc != EXIT_OK )); then
        return "$validation_rc"
    fi

    printf 'Параметры корректны\n'
}

```

Проверяем неверное число:

```

sudo /usr/local/sbin/server-health.sh -w abc
echo $?

```

Ожидается:

```

UNKNOWN: WARNING должен быть числом от 1 до 100
3

```

Проверяем неправильный порядок:

```

sudo /usr/local/sbin/server-health.sh -w 95 -c 90
echo $?

```

Ожидаемый код:

```

3

```

Вывод

Ошибка параметров означает, что проверка не была выполнена корректно. Это состояние UNKNOWN, а не CRITICAL.

Задание 6. Добавить логирование

Добавляем функцию для обычных сообщений:

```
report() {  
    local message=$1  
  
    printf '%s\n' "$message"  
    logger -t server-health -- "$message"  
}
```

Добавляем функцию для ошибок выполнения:

```
report_error() {  
    local message=$1  
  
    printf '%s\n' "$message" >&2  
    logger -p user.err -t server-health -- "$message"  
}
```

Теперь сообщения:

- для мониторинга выводятся в STDOUT;
- ошибки выполнения выводятся в STDERR;
- история запусков сохраняется в Journald.

Проверить отправку тестовой записи:

```
logger -t server-health -- 'Тестовая запись'
```

Посмотреть журнал:

```
journalctl -t server-health -n 10 --no-pager
```

В логах нельзя сохранять:

- пароли;
 - токены;
 - закрытые ключи;
 - содержимое файлов с учётными данными.
-

Задание 7. Реализовать проверку диска

Добавляем функцию:

```

check_disk() {
    local mountpoint=$1
    local warning=$2
    local critical=$3
    local used

    if ! used=$(
        df -P "$mountpoint" 2>/dev/null |
        awk 'NR==2 {
            gsub(/%/,"", $5)
            print $5
        }'
    ); then
        report_error \
            "UNKNOWN: не удалось получить заполнение $mountpoint"
        return "$EXIT_UNKNOWN"
    fi

    if [[ ! $used =~ ^[0-9]+$ ]]; then
        report_error \
            "UNKNOWN: df вернул некорректное значение: $used"
        return "$EXIT_UNKNOWN"
    fi

    if (( used >= critical )); then
        report \
            "CRITICAL: disk $mountpoint usage=${used}%"
        return "$EXIT_CRITICAL"
    fi

    if (( used >= warning )); then
        report \
            "WARNING: disk $mountpoint usage=${used}%"
        return "$EXIT_WARNING"
    fi

    report "OK: disk $mountpoint usage=${used}%"
    return "$EXIT_OK"
}

```

Почему удаляется знак процента

Команда:

```
df -P /
```

возвращает значение вида:

```
31%
```

Строку 31% нельзя корректно сравнивать как число.

В awk выполняется:

```
gsub(/%/,"", $5)
```

После этого функция получает:

31

Проверяем функцию через main():

```
main() {
    parse_arguments "$@"

    local validation_rc=0
    validate_arguments || validation_rc=$?

    if (( validation_rc != EXIT_OK )); then
        return "$validation_rc"
    fi

    check_disk "$MOUNTPPOINT" "$DISK_WARN" "$DISK_CRIT"
}
```

Запускаем:

```
sudo /usr/local/sbin/server-health.sh
echo $?
```

Задание 8. Реализовать проверку службы

Добавляем функцию:

```
check_service() {
    local service=$1

    if systemctl is-active --quiet "$service"; then
        report "OK: service $service is active"
        return "$EXIT_OK"
    fi

    report "CRITICAL: service $service is inactive"
    return "$EXIT_CRITICAL"
}
```

Почему команда помещена внутрь if

При включённом:

```
set -e
```

обычная команда с ненулевым кодом может завершить сценарий.

Неактивная служба является ожидаемым результатом проверки, поэтому команда запускается внутри условия:

```
if systemctl is-active --quiet "$service"; then
```

Проверяем существующую службу:

```
sudo /usr/local/sbin/server-health.sh -s nginx
```

Проверяем несуществующую службу после завершения полной версии сценария:

```
sudo /usr/local/sbin/server-health.sh \  
-s nonexistent.service
```

Ожидаемое состояние службы:

CRITICAL

Задание 9. Собрать полный сценарий

Открываем файл:

```
sudo nano /usr/local/sbin/server-health.sh
```

Приводим его к следующему виду:

```
#!/usr/bin/env bash
```

```
set -euo pipefail
```

```
PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin'  
export PATH
```

```
readonly EXIT_OK=0  
readonly EXIT_WARNING=1  
readonly EXIT_CRITICAL=2  
readonly EXIT_UNKNOWN=3
```

```
SERVICE='nginx'  
MOUNTPPOINT='/'  
DISK_WARN=80  
DISK_CRIT=90
```

```
usage() {  
    cat <<EOF
```

Использование:

```
$0 [-h] [-s SERVICE] [-w WARN] [-c CRIT]
```

Параметры:

-h	показать справку
-s SERVICE	проверяемая systemd-служба

-w WARN порог WARNING в процентах
-c CRIT порог CRITICAL в процентах

Значения по умолчанию:

```
SERVICE=$SERVICE  
MOUNTPPOINT=$MOUNTPPOINT  
WARN=$DISK_WARN  
CRIT=$DISK_CRIT
```

EOF

```
}
```

```
report() {  
    local message=$1  
  
    printf '%s\n' "$message"  
    logger -t server-health -- "$message"  
}
```

```
report_error() {  
    local message=$1  
  
    printf '%s\n' "$message" >&2  
    logger -p user.err -t server-health -- "$message"  
}
```

```
parse_arguments() {  
    local opt  
  
    OPTIND=1  
  
    while getopts ':hs:w:c:' opt; do  
        case "$opt" in  
            h)  
                usage  
                exit "$EXIT_OK"  
                ;;  
            s)  
                SERVICE=$OPTARG  
                ;;  
            w)  
                DISK_WARN=$OPTARG  
                ;;  
            c)  
                DISK_CRIT=$OPTARG  
                ;;  
            :)  
                report_error \  
                    "UNKNOWN: параметр -$OPTARG требует значение"  
                exit "$EXIT_UNKNOWN"  
                ;;  
            \?)
```

```

        report_error \
            "UNKNOWN: неизвестный параметр -$OPTARG"
        usage >&2
        exit "$EXIT_UNKNOWN"
    ;;
esac
done

shift "$((OPTIND - 1))"

if (( $# > 0 )); then
    report_error "UNKNOWN: обнаружены лишние аргументы"
    exit "$EXIT_UNKNOWN"
fi
}

validate_percent() {
    local value=$1

    [[ $value =~ ^[0-9]+$ ]] || return 1
    (( value >= 1 && value <= 100 ))
}

validate_arguments() {
    if [[ -z $SERVICE ]]; then
        report_error \
            'UNKNOWN: имя службы не может быть пустым'
        return "$EXIT_UNKNOWN"
    fi

    if ! validate_percent "$DISK_WARN"; then
        report_error \
            'UNKNOWN: WARNING должен быть числом от 1 до 100'
        return "$EXIT_UNKNOWN"
    fi

    if ! validate_percent "$DISK_CRIT"; then
        report_error \
            'UNKNOWN: CRITICAL должен быть числом от 1 до 100'
        return "$EXIT_UNKNOWN"
    fi

    if (( DISK_WARN >= DISK_CRIT )); then
        report_error \
            'UNKNOWN: WARNING должен быть меньше CRITICAL'
        return "$EXIT_UNKNOWN"
    fi

    return "$EXIT_OK"
}

```

```

check_dependencies() {
    local command_name
    local commands=(
        df
        awk
        systemctl
        hostname
        logger
    )

    for command_name in "${commands[@]}; do
        if ! command -v "$command_name" >/dev/null 2>&1; then
            report_error \
                "UNKNOWN: команда $command_name не найдена"
            return "$EXIT_UNKNOWN"
        fi
    done

    return "$EXIT_OK"
}

```

```

check_disk() {
    local mountpoint=$1
    local warning=$2
    local critical=$3
    local used

    if ! used=$(
        df -P "$mountpoint" 2>/dev/null |
        awk 'NR==2 {
            gsub(/%/,"", $5)
            print $5
        }'
    ); then
        report_error \
            "UNKNOWN: не удалось проверить $mountpoint"
        return "$EXIT_UNKNOWN"
    fi

    if [[ ! $used =~ ^[0-9]+$ ]]; then
        report_error \
            "UNKNOWN: некорректное значение диска: $used"
        return "$EXIT_UNKNOWN"
    fi

    if (( used >= critical )); then
        report \
            "CRITICAL: disk $mountpoint usage=${used}%"
        return "$EXIT_CRITICAL"
    fi
}

```

```

    if (( used >= warning )); then
        report \
            "WARNING: disk $mountpoint usage=${used}%"
        return "$EXIT_WARNING"
    fi

    report "OK: disk $mountpoint usage=${used}%"
    return "$EXIT_OK"
}

check_service() {
    local service=$1

    if systemctl is-active --quiet "$service"; then
        report "OK: service $service is active"
        return "$EXIT_OK"
    fi

    report "CRITICAL: service $service is inactive"
    return "$EXIT_CRITICAL"
}

aggregate_status() {
    local disk_rc=$1
    local service_rc=$2

    if (( disk_rc == EXIT_CRITICAL ||
        service_rc == EXIT_CRITICAL )); then
        report 'SUMMARY: CRITICAL'
        return "$EXIT_CRITICAL"
    fi

    if (( disk_rc == EXIT_UNKNOWN ||
        service_rc == EXIT_UNKNOWN )); then
        report 'SUMMARY: UNKNOWN'
        return "$EXIT_UNKNOWN"
    fi

    if (( disk_rc == EXIT_WARNING ||
        service_rc == EXIT_WARNING )); then
        report 'SUMMARY: WARNING'
        return "$EXIT_WARNING"
    fi

    report 'SUMMARY: OK'
    return "$EXIT_OK"
}

main() {
    parse_arguments "$@"

```

```

local validation_rc=0
local dependency_rc=0
local disk_rc=0
local service_rc=0

validate_arguments || validation_rc=$?

if (( validation_rc != EXIT_OK )); then
    return "$validation_rc"
fi

check_dependencies || dependency_rc=$?

if (( dependency_rc != EXIT_OK )); then
    return "$dependency_rc"
fi

check_disk \
    "$MOUNTPOINT" \
    "$DISK_WARN" \
    "$DISK_CRIT" ||
    disk_rc=$?

check_service "$SERVICE" ||
    service_rc=$?

aggregate_status "$disk_rc" "$service_rc"
}

main "$@"

```

Проверяем синтаксис:

```
sudo bash -n /usr/local/sbin/server-health.sh
```

Проверяем права:

```
sudo chmod 750 /usr/local/sbin/server-health.sh
```

Задание 10. Проверить все коды завершения

1. Состояние OK

Убедиться, что Nginx работает:

```
sudo systemctl start nginx
```

Запустить:

```
sudo /usr/local/sbin/server-health.sh
rc=$?
echo "exit_code=$rc"
```

Ожидаемый код при незаполненном диске:

0

2. Состояние CRITICAL

Проверить несуществующую службу:

```
sudo /usr/local/sbin/server-health.sh \
-s nonexistent.service
rc=$?
echo "exit_code=$rc"
```

Ожидаемый код:

2

3. Состояние UNKNOWN

Передать некорректный порог:

```
sudo /usr/local/sbin/server-health.sh -w abc
rc=$?
echo "exit_code=$rc"
```

Ожидаемый код:

3

4. Состояние WARNING

Смотрим текущее заполнение:

```
df -P / |
awk 'NR==2 {
    gsub(/%/,"", $5)
    print $5
}'
```

Допустим, текущее заполнение равно 35.

Тогда запускаем:

```
sudo /usr/local/sbin/server-health.sh \
-w 35 \
-c 90
rc=$?
echo "exit_code=$rc"
```

Ожидаемый код:

1

Порог WARNING должен быть не больше текущего заполнения, а CRITICAL — больше текущего заполнения.

5. Справка

```
sudo /usr/local/sbin/server-health.sh -h
rc=$?
echo "exit_code=$rc"
```

Ожидаемый код:

0

6. Запуск из другого каталога

```
cd /tmp
sudo /usr/local/sbin/server-health.sh
```

Сценарий не должен зависеть от текущего каталога.

7. Запуск с минимальным окружением

```
sudo env -i \
  PATH='/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin' \
  /usr/local/sbin/server-health.sh
```

8. Проверка журнала

```
journalctl \
  -t server-health \
  -n 30 \
  --no-pager
```

В журнале должны присутствовать результаты запусков.

6. Дополнительные задания

Дополнительное задание 1. Проверить сценарий через ShellCheck

Устанавливаем ShellCheck:

```
sudo apt install -y shellcheck
```

Проверяем сценарий:

```
sudo shellcheck /usr/local/sbin/server-health.sh
```

ShellCheck может обнаружить:

- незакавыченные переменные;
- неправильную работу с \$?;

- подозрительные command substitution;
- ошибки в массивах;
- неиспользуемые переменные.

После исправления повторно выполнить:

```
sudo bash -n /usr/local/sbin/server-health.sh
sudo shellcheck /usr/local/sbin/server-health.sh
```

Дополнительное задание 2. Защитить сценарий от параллельного запуска

В функцию main() после разбора аргументов добавить:

```
exec 9>/run/lock/server-health.lock
```

```
if ! flock -n 9; then
    report_error \
        'UNKNOWN: другой экземпляр сценария уже работает'
    return "$EXIT_UNKNOWN"
fi
```

Добавить flock в список зависимостей:

```
local commands=(
    df
    awk
    systemctl
    hostname
    logger
    flock
)
```

Для проверки временно добавить перед основной логикой:

```
sleep 20
```

В первом терминале запустить сценарий:

```
sudo /usr/local/sbin/server-health.sh
```

Не дожидаясь завершения, запустить во втором терминале:

```
sudo /usr/local/sbin/server-health.sh
echo $?
```

Ожидаемый результат второго экземпляра:

```
UNKNOWN: другой экземпляр сценария уже работает
3
```

После проверки удалить временную команду `sleep 20`.

Дополнительное задание 3. Проверять несколько служб

Убедиться в названиях служб:

```
systemctl list-unit-files |  
  grep -E 'nginx|ssh'
```

Создать массив:

```
SERVICES=(  
    nginx  
    ssh  
)
```

Добавить цикл:

```
for service in "${SERVICES[@]}; do  
    service_rc=0  
    check_service "$service" || service_rc=$?  
  
    if (( service_rc == EXIT_CRITICAL )); then  
        overall_service_rc=$EXIT_CRITICAL  
    fi  
done
```

Обратить внимание на конструкцию:

```
"${SERVICES[@]}"
```

Она передаёт каждый элемент массива отдельно.

При расширении сценария нужно сохранить правило агрегации: одна критичная служба должна давать общий результат `CRITICAL`.

7. Основные команды проверки

Синтаксис и качество кода

```
bash -n /usr/local/sbin/server-health.sh  
shellcheck /usr/local/sbin/server-health.sh
```

Состояние службы

```
systemctl is-active nginx  
systemctl status nginx --no-pager
```

Заполнение диска

```
df -P /  
df -hT /
```

Коды завершения

```
/usr/local/sbin/server-health.sh  
echo $?
```

Журнал

```
journalctl -t server-health --no-pager
```

Права файла

```
ls -l /usr/local/sbin/server-health.sh
```

8. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Назначение сценария `server-health.sh`.
3. Поддерживаемые параметры запуска.
4. Таблицу кодов 0–3.
5. Функции, из которых состоит сценарий.
6. Результат `bash -n`.
7. Результаты запусков OK, WARNING, CRITICAL и UNKNOWN.
8. Пример записи из Journald.
9. Краткий вывод по работе.

Полный текст сценария прикладывается отдельным текстовым файлом или вставляется в приложение к отчёту.

9. Чек-лист выполнения

- ☐ Создан `/usr/local/sbin/server-health.sh`.
- ☐ Используется `#!/usr/bin/env bash`.
- ☐ Включён `set -euo pipefail`.
- ☐ Задан фиксированный PATH.
- ☐ Переменные заключаются в двойные кавычки.
- ☐ Реализованы параметры `-h`, `-s`, `-w` и `-c`.

- ☐ Пороговые значения проверяются.
 - ☐ Реализована функция проверки диска.
 - ☐ Реализована функция проверки службы.
 - ☐ Результаты проверок агрегируются.
 - ☐ Используются коды 0–3.
 - ☐ Ошибки выполнения выводятся в STDERR.
 - ☐ Результаты записываются через `logger`.
 - ☐ `bash -n` не обнаруживает ошибок.
 - ☐ Проверены состояния OK, WARNING, CRITICAL и UNKNOWN.
 - ☐ Сценарий работает из другого каталога.
 - ☐ Сценарий работает в минимальном окружении.
-

10. Контрольные вопросы

1. Чем "\$@" отличается от "\$*"?
2. Почему переменные обычно заключают в двойные кавычки?
3. Почему значение 82% нельзя напрямую сравнивать как число?
4. Чем `return` функции отличается от `exit` сценария?
5. Для чего используются коды 0, 1, 2 и 3?
6. Почему ожидаемо неуспешную команду помещают внутрь `if` или `||` при `set -e`?
7. Что должно выводиться в STDOUT, STDERR и Journald?
8. Почему health-check сценарий не должен автоматически перезапускать службы?